

Swinburne University of Technology*School of Science, Computing and Emerging Technologies***ASSIGNMENT COVER SHEET**

Subject Code: COS30008
Subject Title: Data Structures & Patterns
Assignment number and title: 2 - Iterators
Due date: Sunday, 13 April, 2025, 23:59
Lecturer: Dr. Markus Lumpe

Your name: Chrishelle Anthoneyz **Your student ID:** 104067504

Marker's comments:

Problem	Marks	Obtained
1	44	
2	64	
Total	108	

Extension certification:

This assignment has been given an extension and is now due on _____

Signature of Convener: _____

```
#include "FibonacciSequence.h"
```

```
// Constructor - Initializes the sequence with by setting fPrevious to 0 and fCurrent to 1. ➤
```

```
FibonacciSequence::FibonacciSequence() noexcept  
    : fPrevious(0), fCurrent(1)  
{  
}
```

```
// Operator* - Returns a constant reference to the current Fibonacci value.
```

```
const uint64_t& FibonacciSequence::operator*() const noexcept {  
    return fCurrent;  
}
```

```
// Prefix Increment operator - Advances the Fibonacci sequence.
```

```
FibonacciSequence& FibonacciSequence::operator++() noexcept {  
    uint64_t temp = fCurrent;           // Save the current value  
    fCurrent = fCurrent + fPrevious;    // Calculate the next Fibonacci number  
    fPrevious = temp;                  // Update fPrevious to the old current value  
    return *this;                      // Return the updated object  
}
```

```
// Postfix increment operator
```

```
FibonacciSequence FibonacciSequence::operator++(int) noexcept {  
    FibonacciSequence temp = *this;    // Saves the current state  
    ++(*this);                         // Advances the iterator  
    return temp;                       // Returns the old state  
}
```

```
// Equality operator - Checks if two FibonacciSequence objects are equal by comparing their fPrevious and fCurrent values. ➤
```

```
bool FibonacciSequence::operator==(const FibonacciSequence& other) const ➤  
    noexcept {  
    return (fPrevious == other.fPrevious) && (fCurrent == other.fCurrent);  
}
```

```
// begin - Resets the sequence to its starting values (0 and 1)
```

```
void FibonacciSequence::begin() noexcept {  
    fPrevious = 0;  
    fCurrent = 1;  
}
```

```
// end() - Sets the sequence to a state (fPrevious and fCurrent are zero) that does not occur in valid Fibonacci generation. ➤
```

```
void FibonacciSequence::end() noexcept {  
    fPrevious = 0;  
    fCurrent = 0;  
}
```

```
#include "FibonacciSequenceIterator.h"
```

```
// Constructor - Initializes the iterator by setting fSequence to the provided  
FibonacciSequence pointer and fIndex to the specified starting index.
```

```
FibonacciSequenceIterator::FibonacciSequenceIterator(FibonacciSequence*  
    aSequence, uint64_t aStart) noexcept  
    : fSequence(aSequence), fIndex(aStart)  
{  
}
```

```
// Operator* - Returns a constant reference to the current Fibonacci number by  
dereferencing the underlying FibonacciSequence.
```

```
const uint64_t& FibonacciSequenceIterator::operator*() const noexcept {  
    return **fSequence;  
}
```

```
// Prefix Increment operator - Advances the underlying FibonacciSequence and  
increments the iterator index.
```

```
FibonacciSequenceIterator& FibonacciSequenceIterator::operator++() noexcept {  
    ++(*fSequence);           // Advance the Fibonacci sequence  
    ++fIndex;                 // Increment the index  
    return *this;             // Return the updated iterator  
}
```

```
// Postfix Increment operator - Saves the current iterator state, advances the  
iterator using the prefix operator, and returns the saved state.
```

```
FibonacciSequenceIterator FibonacciSequenceIterator::operator++(int) noexcept {  
    FibonacciSequenceIterator temp = *this; // Save current iterator state  
    ++(*this);                             // Advance to the next state  
    return temp;                           // Return the old state  
}
```

```
// Equality operator - Compares the iterator by checking if both fSequence  
pointers and fIndex values are equal.
```

```
bool FibonacciSequenceIterator::operator==(const FibonacciSequenceIterator&  
    aOther) const noexcept {  
    return (fSequence == aOther.fSequence) && (fIndex == aOther.fIndex);  
}
```

```
// begin - Resets the underlying FibonacciSequence to its initial state and  
returns a new iterator with index 0.
```

```
FibonacciSequenceIterator FibonacciSequenceIterator::begin() const noexcept {  
    fSequence->begin();  
    return FibonacciSequenceIterator(fSequence, 0);  
}
```

```
// end - Returns a new iterator representing the end of the sequence by setting  
fIndex to MAX_FIBONACCI.
```

```
FibonacciSequenceIterator FibonacciSequenceIterator::end() const noexcept {
```

```
    return FibonacciSequenceIterator(fSequence, MAX_FIBONACCI);  
}
```